

NEURAL NETWORKS - EXAMPLES

Natural Language Processing

Example 1

- 4 inputs
- 1 hidden layer with 5 neurons and *ReLU* activation function
- Output layer with 2 neurons with *softmax* activation function

Example 1

- **Input Vector**
- Let's assume a single input sample:

$$\mathbf{x} = \begin{bmatrix} 1.0 \\ 0.5 \\ -1.5 \\ 2.0 \end{bmatrix} \quad (\text{shape: } 4 \times 1)$$

Example 1

- **Weights and Biases**
- **Weights from Input to Hidden Layer:**
- Let's define the weight matrix:

$$\mathbf{W}^{[1]} = \begin{bmatrix} 0.2 & -0.5 & 0.1 & 2.0 \\ -1.5 & 1.3 & 0.7 & -0.8 \\ 0.3 & 0.8 & -0.6 & 1.0 \\ 1.2 & -1.1 & 0.5 & 0.9 \\ 0.7 & 0.2 & -1.4 & 0.3 \end{bmatrix} \quad (\text{shape: } 5 \times 4)$$

Example 1

- Bias vector:

$$\mathbf{b}^{[1]} = \begin{bmatrix} 0.1 \\ -0.3 \\ 0.0 \\ 0.5 \\ -0.2 \end{bmatrix} \quad (\text{shape: } 5 \times 1)$$

Example 1

- Weights from Hidden to Output Layer:

$$\mathbf{W}^{[2]} = \begin{bmatrix} 1.0 & -1.0 & 0.5 & -0.2 & 0.3 \\ -0.5 & 0.6 & -1.2 & 1.5 & -0.7 \end{bmatrix} \quad (\text{shape: } 2 \times 5)$$

- Bias vector:

$$\mathbf{b}^{[2]} = \begin{bmatrix} 0.05 \\ -0.05 \end{bmatrix} \quad (\text{shape: } 2 \times 1)$$

Example 1

- **Hidden Layer Calculation**
- **Step 1: Linear Transformation**
- $\mathbf{z}^{[1]} = \mathbf{W}^{[1]} \cdot \mathbf{x} + \mathbf{b}^{[1]}$

Matrix multiplication:

$$\mathbf{z}^{[1]} = \begin{bmatrix} 0.2 & -0.5 & 0.1 & 2.0 \\ -1.5 & 1.3 & 0.7 & -0.8 \\ 0.3 & 0.8 & -0.6 & 1.0 \\ 1.2 & -1.1 & 0.5 & 0.9 \\ 0.7 & 0.2 & -1.4 & 0.3 \end{bmatrix} \cdot \begin{bmatrix} 1.0 \\ 0.5 \\ -1.5 \\ 2.0 \end{bmatrix} + \begin{bmatrix} 0.1 \\ -0.3 \\ 0.0 \\ 0.5 \\ -0.2 \end{bmatrix}$$

Example 1

- Compute the dot product for each row:

- First row:

$$0.2(1.0) + (-0.5)(0.5) + 0.1(-1.5) + 2.0(2.0) = 0.2 - 0.25 - 0.15 + 4.0 = 3.8$$

$$\text{Then add bias: } 3.8 + 0.1 = 3.9$$

- Second row:

$$-1.5 + 0.65 - 1.05 - 1.6 = -3.5, \text{ then } -3.5 - 0.3 = -3.8$$

- Third row:

$$0.3 + 0.4 + 0.9 + 2.0 = 3.6, \text{ then } 3.6 + 0.0 = 3.6$$

- Fourth row:

$$1.2 - 0.55 - 0.75 + 1.8 = 1.7, \text{ then } 1.7 + 0.5 = 2.2$$

- Fifth row:

$$0.7 + 0.1 + 2.1 + 0.6 = 3.5, \text{ then } 3.5 - 0.2 = 3.3$$

So:

$$\mathbf{z}^{[1]} = \begin{bmatrix} 3.9 \\ -3.8 \\ 3.6 \\ 2.2 \\ 3.3 \end{bmatrix}$$

Example 1

- **Step 2: ReLU Activation**
- Apply ReLU:

$$\mathbf{a}^{[1]} = \text{ReLU}(\mathbf{z}^{[1]}) = \max(0, \mathbf{z}^{[1]}) = \begin{bmatrix} 3.9 \\ 0 \\ 3.6 \\ 2.2 \\ 3.3 \end{bmatrix}$$

Example 1

- **Output Layer Calculation**

- **Step 1: Linear Transformation**

- $\mathbf{z}^{[2]} = \mathbf{W}^{[2]} \cdot \mathbf{a}^{[1]} + \mathbf{b}^{[2]}$

- First row:

- $1.0(3.9) + (-1.0)(0) + 0.5(3.6) + (-0.2)(2.2) + 0.3(3.3) = 3.9 + 0 + 1.8 - 0.44 + 0.99 = 6.25, 6.25 + 0.05 = 6.3$

- Second row:

- $-0.5(3.9) + 0.6(0) + (-1.2)(3.6) + 1.5(2.2) + (-0.7)(3.3) = -1.95 + 0 - 4.32 + 3.3 - 2.31 = -5.28, -5.28 - 0.05 = -5.33$

$$\mathbf{z}^{[2]} = \mathbf{W}^{[2]} \cdot \mathbf{a}^{[1]} + \mathbf{b}^{[2]}$$

$$\begin{bmatrix} 1.0 & -1.0 & 0.5 & -0.2 & 0.3 \\ -0.5 & 0.6 & -1.2 & 1.5 & -0.7 \end{bmatrix} \cdot \begin{bmatrix} 3.9 \\ 0 \\ 3.6 \\ 2.2 \\ 3.3 \end{bmatrix} + \begin{bmatrix} 0.05 \\ -0.05 \end{bmatrix}$$

$$\mathbf{z}^{[2]} = \begin{bmatrix} 6.3 \\ -5.33 \end{bmatrix}$$

Example 1

- **Step 2: Softmax Activation**

- $\text{Softmax}(\mathbf{z}^{[2]}) = \frac{e^{z_i}}{\sum_j e^{z_j}}$

- Compute exponentials (subtract max for numerical stability):

- Let's shift by max value (6.3):

- $\mathbf{z}' = \begin{bmatrix} 0 \\ -11.63 \end{bmatrix} \Rightarrow e^{\mathbf{z}'} = \begin{bmatrix} 1 \\ e^{-11.63} \approx 8.9 \times 10^{-6} \end{bmatrix}$

- Sum: $1 + 8.9 \times 10^{-6} \approx 1.0000089$

- $\text{Softmax Output} = \begin{bmatrix} \frac{1}{1.0000089} \\ \frac{8.9 \times 10^{-6}}{1.0000089} \end{bmatrix} \approx \begin{bmatrix} 0.9999911 \\ 0.0000089 \end{bmatrix}$

Example 1

- **Output of the Network**

- $\mathbf{a}^{[2]} = \begin{bmatrix} 0.9999911 \\ 0.0000089 \end{bmatrix}$

- This means the network is highly confident in class 1 (assuming class indices start from 1).

Example 2

- 4 inputs with embedding size 3
- 1 hidden layer with 5 neurons and *ReLU* activation function
- Output layer with 2 neurons with *softmax* activation function

Example 2

- **Input matrix (before flattening):**

- $\mathbf{X} = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \\ 1 & 0 & 1 \end{bmatrix}$

- **Flattened input vector $\mathbf{x} \in \mathbb{R}^{12}$:**

- $\mathbf{x} = [1 \ 2 \ 3 \ 4 \ 5 \ 6 \ 7 \ 8 \ 9 \ 1 \ 0 \ 1]$

Example 2

- **Hidden Layer**

- **Weight matrix** $W^{(1)} \in \mathbb{R}^{5 \times 12}$:

- $W^{(1)} = \begin{bmatrix} 1 & 0 & -1 & 1 & 0 & -1 & 1 & 0 & -1 & 1 & 0 & -1 \\ 0 & 1 & 1 & 0 & 1 & 1 & 0 & 1 & 1 & 0 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \\ 2 & -1 & 0 & 2 & -1 & 0 & 2 & -1 & 0 & 2 & -1 & 0 \\ 0 & 0 & 1 & 0 & 0 & 1 & 0 & 0 & 1 & 0 & 0 & 1 \end{bmatrix}$

- **Bias vector** $b^{(1)} \in \mathbb{R}^5$:

- $b^{(1)} = [0 \quad 1 \quad 0 \quad 2 \quad -1]$

Example 2

- **Linear transformation at hidden layer**
- We compute:
- $\mathbf{z}^{(1)} = W^{(1)} \cdot \mathbf{x} + b^{(1)}$

Example 2

- **Matrix multiplication** $W^{(1)} \cdot \mathbf{x} + \mathbf{b}$

$$\bullet \mathbf{z}^{(1)} = \begin{bmatrix} 1 & 0 & -1 & 1 & 0 & -1 & 1 & 0 & -1 & 1 & 0 & -1 \\ 0 & 1 & 1 & 0 & 1 & 1 & 0 & 1 & 1 & 0 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \\ 2 & -1 & 0 & 2 & -1 & 0 & 2 & -1 & 0 & 2 & -1 & 0 \\ 0 & 0 & 1 & 0 & 0 & 1 & 0 & 0 & 1 & 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} 1 \\ 2 \\ 3 \\ 4 \\ 5 \\ 6 \\ 7 \\ 8 \\ 9 \\ 1 \\ 0 \\ 1 \end{bmatrix} + \begin{bmatrix} 0 \\ 1 \\ 0 \\ 2 \\ -1 \end{bmatrix}$$

Example 2

- Compute each row (dot product with input vector):

- **Row 1:**

- $(1)(1) + (0)(2) + (-1)(3) + (1)(4) + (0)(5) + (-1)(6) + (1)(7) + (0)(8) + (-1)(9) + (1)(1) + (0)(0) + (-1)(1) = 1 - 3 + 4 - 6 + 7 - 9 + 1 - 1 = -6$

- **Add bias:** $-6 + 0 = -6$

- **Row 2:**

- $(0)(1) + (1)(2) + (1)(3) + (0)(4) + (1)(5) + (1)(6) + (0)(7) + (1)(8) + (1)(9) + (0)(1) + (1)(0) + (1)(1) = 2 + 3 + 5 + 6 + 8 + 9 + 1 = 34$

- **Add bias:** $34 + 1 = 35$

Example 2

- **Row 3:**

- All weights are 1:

- $1 + 2 + 3 + 4 + 5 + 6 + 7 + 8 + 9 + 1 + 0 + 1 = 47$

- Add bias: $57 + 0 = 47$

- **Row 4:**

- $(2)(1) + (-1)(2) + (0)(3) + (2)(4) + (-1)(5) + (0)(6) + (2)(7) + (-1)(8) + (0)(9) + (2)(1) + (-1)(0) + (0)(1) = 2 - 2 + 0 + 8 - 5 + 0 + 14 - 8 + 0 + 2 + 0 + 0 = 11$

- Add bias: $11 + 2 = 13$

Example 2

- **Row 5:**

- $(0)(1) + (0)(2) + (1)(3) + (0)(4) + (0)(5) + (1)(6) + (0)(7) + (0)(8) + (1)(9) + (0)(1) + (0)(0) + (1)(1) = 3 + 6 + 9 + 1 = 19$

- **Add bias:** $19 - 1 = 18$

- **Linear output vector before activation $\mathbf{z}^{(1)}$:**

- $\mathbf{z}^{(1)} = \begin{bmatrix} -6 \\ 35 \\ 47 \\ 13 \\ 18 \end{bmatrix}$

Example 2

- **Apply ReLU activation**

- $a(1) = \text{ReLU}(z(1)) = \max(0, z(1))$

- $\mathbf{a}^{(1)} = \begin{bmatrix} 0 \\ 35 \\ 47 \\ 13 \\ 18 \end{bmatrix}$

Example 2

- **Output Layer**
- **Weight matrix** $W^{(2)} \in \mathbb{R}^{2 \times 5}$:
- $W^{(2)} = \begin{bmatrix} 1 & 1 & 0 & 0 & -1 \\ -1 & 0 & 1 & 1 & 1 \end{bmatrix}$
- **Bias vector** $b^{(2)} \in \mathbb{R}^2$:
- $b^{(2)} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$

Example 2

- **Linear transformation at output layer**

- $\mathbf{z}^{(2)} = W^{(2)} \cdot \mathbf{a}^{(1)} + b^{(2)}$

- Perform matrix multiplication:

- $$\mathbf{z}^{(2)} = \begin{bmatrix} 1 & 1 & 0 & 0 & -1 \\ -1 & 0 & 1 & 1 & 1 \end{bmatrix} \cdot \begin{bmatrix} 0 \\ 35 \\ 47 \\ 13 \\ 18 \end{bmatrix} + \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

Example 2

- **Output neuron 1:**

- $(1)(0) + (1)(35) + (0)(47) + (0)(13) + (-1)(18) = 35 - 18 = 17$

- **Output neuron 2:**

- $(-1)(0) + (0)(35) + (1)(47) + (1)(13) + (1)(18) = 57 + 13 + 18 = 78$

- **Linear output of output layer:**

- $\mathbf{z}^{(2)} = \begin{bmatrix} 17 \\ 78 \end{bmatrix}$

Example 2

- **Apply softmax**

- $\text{softmax}(z_i) = \frac{e^{z_i}}{\sum_j e^{z_j}}$

- Apply numerical stability trick: subtract max (78):

- $\mathbf{z}^{(2)} - \max = \begin{bmatrix} -61 \\ 0 \end{bmatrix}$

- $\text{softmax}\left(\begin{bmatrix} -61 \\ 0 \end{bmatrix}\right) = \begin{bmatrix} \frac{e^{-61}}{e^{-61} + e^0} \\ \frac{e^0}{e^{-61} + e^0} \end{bmatrix} \approx \begin{bmatrix} 0 \\ 1 \end{bmatrix}$

- **Final Output of Network**

- $\begin{bmatrix} 0 \\ 1 \end{bmatrix}$

- The network predicts **class 2 (index 1)** with full confidence.

References

- https://www.youtube.com/watch?v=sNTtUV9yE_M